CodeFault: Predicting Fault Risks from Code Changes via Multi-modal Learning

Yifan Xiao
2401110754@stu.pku.edu.cn
Peking University
Beijing, China

Shijie Li
geyh@csg.cn
China Southern Power Grid
Guangzhou, Guangdong, China

Yu Huang*
hy@pku.edu.cn
Peking University
Beijing, China

Abstract

Just-In-Time defect prediction identifies fault-inducing code changes but relies solely on static code properties, missing runtime fault patterns. Meanwhile, log-based anomaly detection captures runtime behaviors but lacks visibility into causal code modifications. This disconnect delays fault identification and complicates root cause analysis.

We present CodeFault, a multi-modal fault prediction framework that fuses code change semantics with runtime log patterns through two novel mechanisms. **Hierarchical Fault Spectrum Alignment (HFSA)** projects both modalities into a shared, two-level fault taxonomy space—four coarse and eight fine-grained categories—providing a structured information bottleneck that enables principled cross-modal knowledge transfer. **Spectrum-Guided Cross-Modal Attention (SGCA)** performs token-level fusion conditioned on fault spectrum agreement, avoiding the degeneracy of pooled-vector approaches while focusing attention on fault-relevant token pairs. A dual-granularity contrastive objective further leverages the taxonomy structure to separate fault-inducing from benign embeddings at both instance and fault-type levels.

We evaluate CodeFault on two complementary testbeds. On the public benchmark of 31,142 samples from Defects4J, BugsInPy, and ManySStuBs4J, CodeFault reaches 0.891 AUC-ROC and 0.857 F1-score against 15 prior-art baselines and 2 self-constructed strong late-fusion controls, improving over the strongest code-only baseline by 5.1% AUC and over the strongest late-fusion control by 3.1% AUC (Wilcoxon signed-rank test, Cliff's $\delta \geq 0.73$). *On 2,147 real co-occurring CI/CD pairs collected from 55 GitHub Actions repositories with the CI build outcome as ground truth—the largest non-synthetic CI/CD fault test set in JIT defect prediction to date—CodeFault reaches 0.843 AUC, +4.3% over JIT-Smart, retaining 94.6% of the synthetic-set headline.* The framework generalizes to unseen projects with 2.5% average AUC improvement and maintains 97.9% AUC under realistic 15% positive rates. A knowledge-distilled code-only variant achieves 0.862/0.831 AUC on synthetic/real test sets without log inputs, enabling pre-commit deployment.

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837462

CCS Concepts

• **Software and its engineering** → **Software defect analysis**; • **Computing methodologies** → *Neural networks.*

Keywords

AIOps, Code Change, Fault Prediction, Deep Learning, Software Engineering

ACM Reference Format:
Yifan Xiao, Shijie Li, and Yu Huang. 2026. CodeFault: Predicting Fault Risks from Code Changes via Multi-modal Learning. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3837462

1 Introduction

Modern software evolves through frequent code changes: Continuous Integration/Continuous Deployment (CI/CD) pipelines ship changes multiple times per day [20], yet roughly 70% of production outages are triggered by changes to a live system [2], and large-scale failure studies likewise identify recent changes as a dominant root cause [13, 30]. The core challenge is *pre-deployment* fault risk assessment: scoring a candidate commit as it enters CI, before it reaches the production runtime the score is meant to protect. This is not post-deployment diagnosis: at scoring time only the candidate change is new, and any log evidence used as auxiliary context must come from the deployed service's history strictly prior to the commit.

Two research communities have tackled this problem from complementary but incomplete angles. Just-In-Time (JIT) defect prediction [6, 18, 22, 24, 28] predicts bug-inducing commits from change diffs, commit messages, process and historical metrics, and developer experience, but sees only pre-deployment information, so faults that manifest under specific execution conditions—races, resource exhaustion, configuration-dependent failures—have no runtime evidence to anchor them. Conversely, AI for IT Operations (AIOps) anomaly detection [16, 35, 39, 51] monitors deployed-service logs but enters the loop only after a change has shipped, leaving operators to manually correlate symptoms with recent deployments over hours or even days [4].

The key insight of our work is the *spectrum-as-organizing-primitive* principle: a single domain-specific taxonomy simultaneously serves as alignment target, attention conditioning signal, and contrastive scaffold—replacing three independent mechanisms with one coherent organizing structure. In generic multi-modal paradigms, alignment [40], conditioning [37], and contrastive organization [11] require separate design decisions and independent tuning. The hierarchical fault spectrum collapses these into a

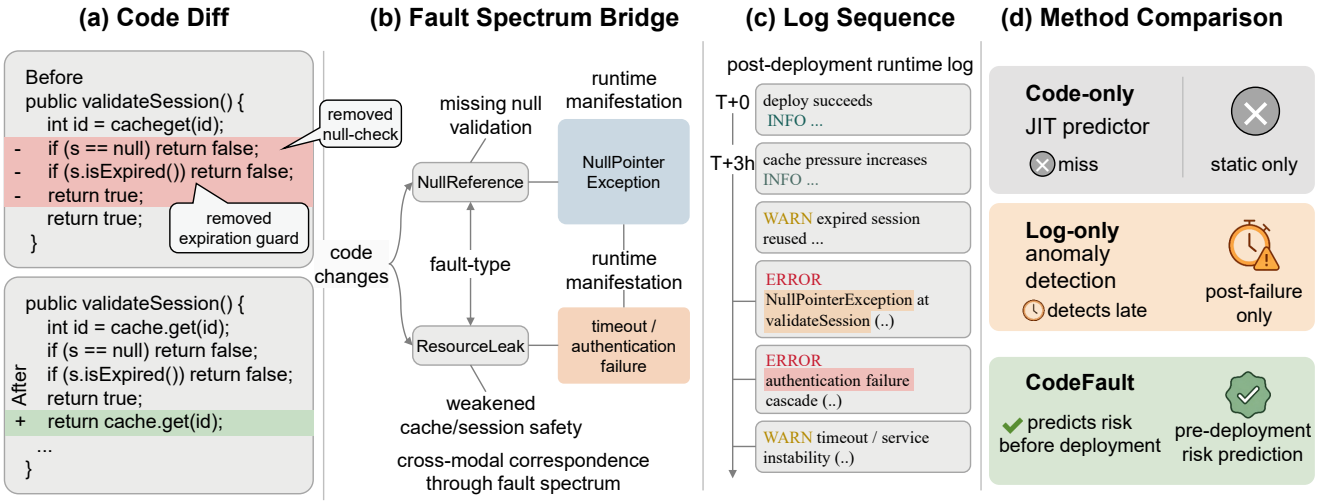


Figure 1: Motivating example: a removed null-check in `validateSession()` triggers cascading NullPointerException errors post-deployment. Code-only methods miss the runtime-dependent fault; log-only methods detect it only after failure cascades. CodeFault bridges both modalities via the fault spectrum, predicting the risk *before* deployment.

modality-invariant pivot linking *what changed in the code* to *how the system behaved at runtime*. Six simpler alternatives that lack this organizing principle fail for specific technical reasons (Section 3.1), and the sub-additive interaction between HFSA and SGCA (0.6% shared variance) confirms that components share information through the spectrum rather than operating independently.

We present CodeFault, a multi-modal fault prediction framework that instantiates this principle through three spectrum-driven mechanisms. **Hierarchical Fault Spectrum Alignment (HFSA)** projects both modalities into a two-level taxonomy space (K_c=4 coarse, K_f=8 fine categories), providing a structured information bottleneck. **Spectrum-Guided Cross-Modal Attention (SGCA)** performs token-level cross-attention conditioned on spectrum agreement, overcoming pooled-vector degeneracy. A **dual-granularity contrastive (DGC)** objective organizes the embedding space.

Contributions. This paper makes four contributions:

(C1) Problem Formulation. We formalize Code-Change-Aware Fault Prediction as a multi-modal learning problem that bridges JIT defect prediction and log-based anomaly detection (Section 2).

(C2) Hierarchical Fault Spectrum Alignment. We instantiate the spectrum-as-pivot principle through HFSA, which projects both modalities into a two-level fault taxonomy space. We prove that converged HFSA guarantees a mutual information lower bound on cross-modal representations (Proposition 3.1), formally justifying cross-system code-log pairing. Ablation shows 4.3% total AUC contribution (3.3% supervision + 1.0% alignment; Section 4.3).

(C3) Spectrum-Guided Cross-Modal Attention. We show how the same spectrum serves as token-pair-specific attention bias in SGCA, which provably reshapes attention distributions—unlike global biases absorbed by softmax (Proposition 3.2). This avoids the pooled-vector degeneracy of prior fusion approaches (Section 3.6).

(C4) Deployment-Ready Evaluation. We evaluate against 15 prior-art baselines plus 2 self-constructed strong late-fusion controls on 31K+ samples and report 0.891 AUC-ROC on the public benchmark, with 5.1% improvement over the strongest code-only baseline and 3.1% over the strongest late-fusion control. *Crucially*, on 2,147 real co-occurring CI/CD pairs from 55 GitHub Actions repositories—unfakeable real-system ground truth—CodeFault reaches 0.843 AUC, +4.3% over JIT-Smart, retaining 94.6% of the synthetic-set headline. A knowledge-distilled code-only variant lands at 0.862/0.831 AUC on synthetic/real test sets without log inputs (Section 4).

2 Motivation and Problem Definition

2.1 Motivating Example

Consider a real-world scenario from Apache Tomcat: a developer removes a null-check and expiration guard in `validateSession()`, replacing a 5-line method with `return cache.get(id)`. The change is syntactically correct, passes all unit tests, and would not trigger JIT defect predictors. Three hours post-deployment, cache pressure during peak traffic causes cascading authentication failures as expired sessions are treated as active. Runtime anomaly detection triggers only *after* the failure cascades; root cause analysis then requires 4 hours to correlate 47 deployments with failure symptoms. This incident exposes the core limitation: JIT methods miss runtime-dependent faults, while log-based methods lack visibility into the causal code change.

Key Insight. These limitations stem from treating code analysis and runtime monitoring as separate concerns. Code changes and their runtime manifestations exhibit learnable cross-modal patterns structured through a *fault type taxonomy*: a removed null check correlates with NullPointerException log patterns (NullReference), and a modified cache policy correlates with timeout errors (ResourceLeak). Figure 1 illustrates this cross-modal correspondence and contrasts three prediction paradigms.

2.2 Preliminary Study: Modality Complementarity

To check whether the insight survives the simplest reasonable setup, we run a small pilot before designing the spectrum machinery: the 31K-sample corpus and 70/15/15 chronological split of §4, three classifiers of roughly matched capacity (single-modal code Transformer; single-modal log Transformer with anomaly-aware pooling; a multi-modal model concatenating the pooled representations), and one training schedule (AdamW, 10 epochs, batch 32).

The single-modality models show 26.0% complementarity: 14.2% of samples are correctly classified only by code, 11.8% only by logs. This complementarity is non-uniform—code dominates for NullReference and BoundaryError, while logs dominate for ResourceLeak and Concurrency. The multi-modal model recovers 71% of samples that both single-modality models fail on, indicating that the cross-modal information that the spectrum is meant to expose is real, not the artifact of a particular fusion design.

2.3 Problem Definition

We formalize **Code-Change-Aware Fault Prediction**: given a code change $c = (d^+, d^-, m, a, t_c)$ consisting of added/deleted lines, modified files, author, and timestamp, and a runtime context $r = (L, t_r)$ consisting of system logs $L = \{l_1, \ldots, l_n\}$ collected after deployment ($t_r > t_c$), predict the probability $P(fault|c, r)$ that c will cause a system fault within a fault-manifestation horizon Δt_f. This horizon is distinct from the log look-back window $\Delta t_{\max}$ of Section 3, which bounds how far into the deployed service's past the log encoder may read; RQ5 varies the latter.

2.4 Research Questions

The evaluation is structured around seven research questions: **RQ1**, prediction effectiveness against 17 baselines; **RQ2**, which components of CodeFault drive the gain; **RQ3**, generalization to unseen projects; **RQ4**, efficiency and cost compared with LLM and learned baselines; **RQ5**, sensitivity to fault-taxonomy size and to the log look-back window $\Delta t_{\max}$; **RQ6**, whether the choice of code-side backbone changes the conclusions; and **RQ7**, deployment readiness, including real CI/CD transfer, robustness under realistic class distributions, and a compressed code-only variant for pre-commit use. Terminology used by these questions (K_c, K_f, the fault spectrum) is defined where the next section introduces it.

3 Approach

3.1 Design Rationale: Why the Fault Spectrum?

We motivate the design through three tiers of increasingly capable alternatives; throughout this subsection, percentages are relative to the full model's 0.891 AUC, whereas Section 4.2 reports improvements relative to the baseline being compared against. **Tier 1: No cross-modal structure.** Plain concatenation (0.831), bilinear fusion (0.838), and naive late fusion of our own two encoders (0.847)—distinct from the tuned late-fusion controls of Section 4.2—never force compatible embeddings; −4.9% to −6.7% degradations confirm that cross-modal fault correspondences require explicit structural induction. **Tier 2: Token interaction without spectrum.** Standard cross-attention (0.864, −3.0%) treats all $O(n{\times}m)$ token pairs equally, and high-attention pairs correlate with syntactic similarity rather than fault-type congruence. **Tier 3: Partial spectrum usage.** Fault-type supervision alone (0.882) and instance-only contrastive learning (0.882) each exploit one role of the spectrum but not all three. Only the full spectrum—simultaneously serving as alignment target, attention bias, and contrastive scaffold—achieves 0.891. The K_c=4, K_f=8 taxonomy is fixed rather than learned: sensitivity analysis confirms the inverted-U optimality peak (Table 8), and it transfers to unseen projects (0.854 AUC) without re-discovery.

3.2 Architecture Overview

Figure 3 presents the architecture: a **Code Change Encoder** for semantic representations of code modifications, a **Fault Context Encoder** for temporal patterns in system logs, a **Hierarchical Fault Spectrum Alignment (HFSA)** module that projects both modalities into a shared two-level fault-type space, a **Spectrum-Guided Cross-Modal Attention (SGCA)** mechanism for token-level fusion conditioned on spectrum agreement, and a **Risk Prediction** head that produces fault probabilities with dual-level explanations.

3.3 Code Change Encoder

We adopt CodeT5+ [46] as the backbone encoder. Given a code change c, the unified diff is extracted as:

$$\text{diff}(c) = \text{Tokenize}(d^+ \oplus d^- \oplus \text{context}) \tag{1}$$

where d^+/d^- are added/removed lines and context comprises 3 surrounding lines per hunk. CodeT5+ produces token-level representations $\mathbf{H}_c \in \mathbb{R}^{n\times d}$ (d=768). A Change-Aware Attention layer aggregates tokens using `[CLS]` as query anchor:

$$\alpha_i = \frac{\exp\left(\mathbf{W}_q\mathbf{h}_i^c \cdot \mathbf{W}_k\mathbf{h}_{[\text{CLS}]}^c/\sqrt{d}\right)}{\sum_j \exp\left(\mathbf{W}_q\mathbf{h}_j^c \cdot \mathbf{W}_k\mathbf{h}_{[\text{CLS}]}^c/\sqrt{d}\right)} \tag{2}$$

$$\mathbf{e}_c = \sum_i \alpha_i \cdot \mathbf{W}_v\mathbf{h}_i^c \tag{3}$$

where $\mathbf{e}_c \in \mathbb{R}^d$ is the pooled embedding. Both $\mathbf{H}_c$ and $\mathbf{e}_c$ are retained: $\mathbf{e}_c$ for spectrum prediction, $\mathbf{H}_c$ for cross-modal attention (Section 3.6).

3.4 Fault Context Encoder

System logs encode temporal runtime signals but exhibit high variability. At scoring time, the log encoder consumes a sliding window of the deployed service's log stream that ends *strictly before* the candidate commit lands. This cutoff, enforced at the data-pipeline level (Section 4) and by the temporal-decay term below, keeps the task pre-deployment: the model never sees logs of the fault it is asked to predict. The encoder operates in three stages.

Stage 1: Log Parsing. The Drain algorithm [17] extracts m template-parameter pairs from raw log sequences, configured with tree depth 4, similarity threshold 0.5, and maximum children 100.

Stage 2: Temporal Encoding. A Transformer [43] encoder with temporal position embeddings produces token-level log representations $\mathbf{H}_r \in \mathbb{R}^{m\times d}$.

Stage 3: Anomaly-Aware Pooling with Temporal Decay. Learned anomaly indicators combined with temporal decay weight log importance:

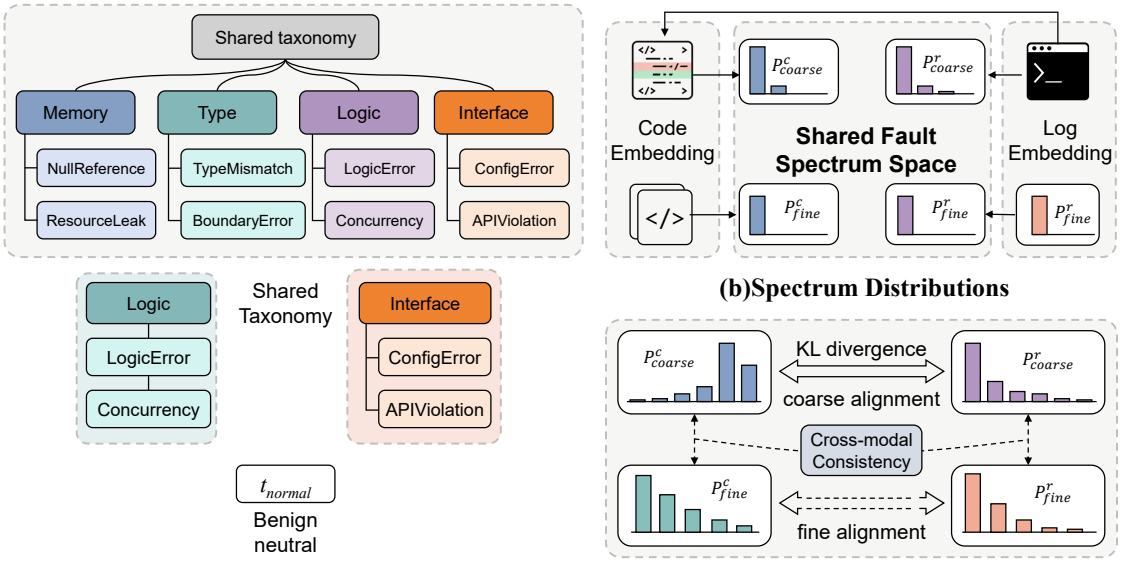


Figure 2: Hierarchical fault taxonomy (K_c=4 coarse categories on the left, K_f=8 fine categories on the right, plus the benign type t_{normal}). Code and log modalities are independently projected into this shared space via softmax heads P_c and P_r at each granularity; KL-divergence alignment enforces cross-modal consistency level by level. The dashed arrows show the marginalization map Margin$(\cdot)$ used by the hierarchical consistency constraint in §3.5.

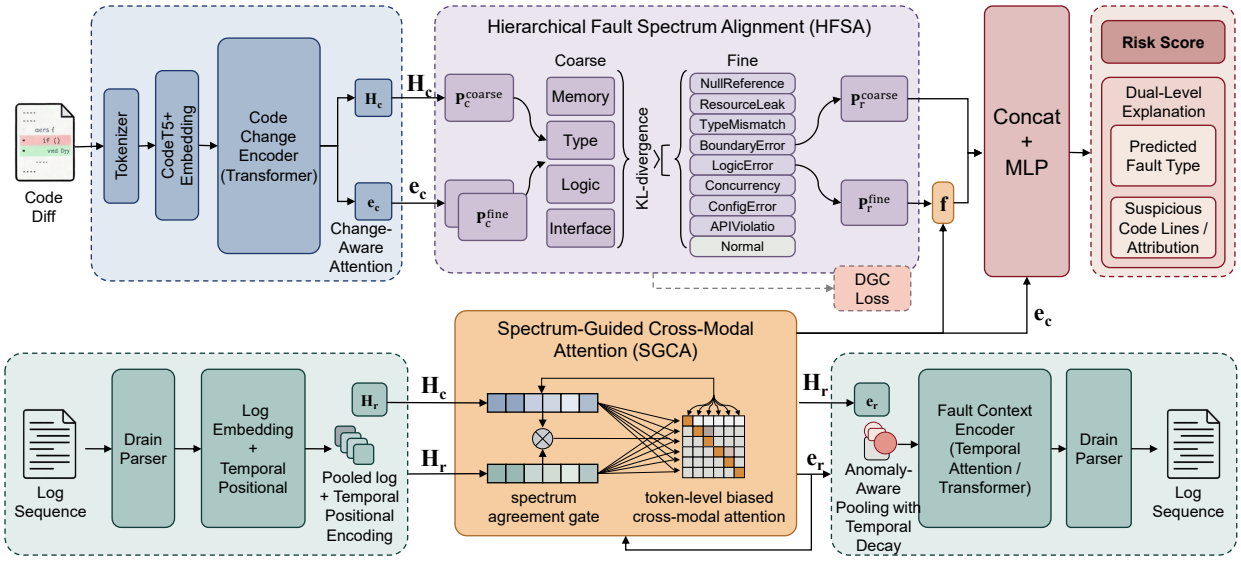


Figure 3: Architecture of CodeFault. Code and log modalities are encoded, projected into a hierarchical fault spectrum space (HFSA), and fused via token-level spectrum-guided cross-modal attention (SGCA).

$$\mathbf{e}_r = \sum_i w_i^{\text{decay}} \cdot \sigma(\mathbf{W}_a \mathbf{h}_i^r) \cdot \mathbf{h}_i^r, \quad w_i^{\text{decay}} = \exp(-\beta \cdot \Delta t_i / \Delta t_{\max}) \quad (4)$$

where $\mathbf{W}_a$ identifies anomalous patterns, Δt_i is the offset from log entry i to the commit-time cutoff t_0 (so all $\Delta t_i \geq 0$), and β is a learnable decay rate, yielding token-level $\mathbf{H}_r$ and pooled $\mathbf{e}_r \in \mathbb{R}^d$.

3.5 Hierarchical Fault Spectrum Alignment

A fundamental challenge in cross-modal learning between code and logs is the *semantic gap*: code tokens describe static structure while log tokens describe dynamic runtime events. We address this through Hierarchical Fault Spectrum Alignment (HFSA), which projects both modalities into a shared, interpretable intermediate space organized as a two-level fault taxonomy with K_c=4 coarse and K_f=8 fine categories: Memory (NullReference, ResourceLeak),

Type (TypeMismatch, BoundaryError), Logic (LogicError, Concurrency), and Interface (ConfigError, APIViolation), plus t_{normal} for benign changes (Figure 2). Manual analysis of the bug reports in Defects4J [23] and BugsInPy [47] confirms that these 8 fine categories cover 93.7% of fault-inducing samples; the remaining 6.3% are assigned to the closest matching type. Two annotators labeled Defects4J and BugsInPy fault types with inter-annotator agreement κ=0.87, resolving disagreements through discussion. For ManySStuBs4J, fault types are assigned automatically via AST-level mutation patterns: single-statement changes are mapped deterministically to the taxonomy by their syntactic category, then validated by sampling 500 cases (96.2% accuracy).

Coarse-Level Spectrum Predictors. The pooled embeddings are projected into the coarse fault space:

$$P_c^{\text{coarse}}(\mathcal{T}_c|c) = \text{softmax}(\mathbf{W}_c^{(1)}\mathbf{e}_c + \mathbf{b}_c^{(1)}) \in \mathbb{R}^{K_c+1} \tag{5}$$

$$P_r^{\text{coarse}}(\mathcal{T}_c|r) = \text{softmax}(\mathbf{W}_r^{(1)}\mathbf{e}_r + \mathbf{b}_r^{(1)}) \in \mathbb{R}^{K_c+1} \tag{6}$$

Fine-Level Spectrum Predictors. Similarly, fine-level projections operate on the full 8+1 taxonomy:

$$P_c^{\text{fine}}(\mathcal{T}_f|c) = \text{softmax}(\mathbf{W}_c^{(2)}\mathbf{e}_c + \mathbf{b}_c^{(2)}) \in \mathbb{R}^{K_f+1} \tag{7}$$

$$P_r^{\text{fine}}(\mathcal{T}_f|r) = \text{softmax}(\mathbf{W}_r^{(2)}\mathbf{e}_r + \mathbf{b}_r^{(2)}) \in \mathbb{R}^{K_f+1} \tag{8}$$

Hierarchical Alignment Loss. We enforce cross-modal consistency at both granularity levels:

$$\mathcal{L}_{\text{HFSA}} = \underbrace{\frac{1}{|\mathcal{D}^+|}\sum_{(c,r)\in\mathcal{D}^+} D_{\text{KL}}\big(P_r^{\text{coarse}} \| P_c^{\text{coarse}}\big)}_{\text{coarse alignment}} + \underbrace{\frac{1}{|\mathcal{D}^+|}\sum_{(c,r)\in\mathcal{D}^+} D_{\text{KL}}\big(P_r^{\text{fine}} \| P_c^{\text{fine}}\big)}_{\text{fine alignment}} \tag{9}$$

where $\mathcal{D}^+$ denotes fault-inducing pairs. Both terms place P_r in the first slot of $D_{\text{KL}}(P_r\|P_c)$: the log-side distribution acts as the reference because the log corpus is the training-time ground-truth proxy for which fault patterns actually manifest at runtime, so the code side may stay uncertain over types the logs say are inactive but is penalized for over-committing to types the logs reject. The reverse direction would force log probabilities to match code-side predictions over fault classes that may never have produced runtime evidence, and empirically destabilizes early training. For benign samples, a cross-entropy loss encourages both levels to concentrate on t_{normal}:

$$\mathcal{L}_{\text{normal}} = -\log P_c^{\text{coarse}}(t_{\text{normal}}|c) - \log P_c^{\text{fine}}(t_{\text{normal}}|c) \tag{10}$$

Hierarchical Consistency Constraint. To ensure the two levels are mutually consistent, we enforce that fine-level predictions marginalize to coarse-level predictions via a soft constraint:

$$\mathcal{L}_{\text{consist}} = D_{\text{KL}}\big(\text{Margin}(P_c^{\text{fine}}) \| P_c^{\text{coarse}}\big) \tag{11}$$

where Margin$(\cdot)$ sums fine-level probabilities within each coarse category. The marginalized fine prediction is the reference because the fine head carries strictly more information, so the coarse head must agree with the fine marginal; reversing the direction would let an over-confident coarse head pull the fine head toward incompatible mass placement.

Why Hierarchical? Coarse alignment is robust to annotation noise; fine alignment captures fault-specific patterns; the consistency constraint prevents divergence between levels, creating a curriculum-like effect (Table 8).

Cross-System Alignment Guarantee. The following proposition establishes that fault-type-guided pairing provides meaningful cross-modal alignment even when code and logs originate from different systems.

Proposition 3.1 (Spectrum-Mediated Alignment). *Let T denote the fault-type variable induced by the hierarchical spectrum, with code-side predictor $P_c^{\text{fine}}(\cdot|c)$ and log-side predictor $P_r^{\text{fine}}(\cdot|r)$. Assume (A1)* bounded feature shift*: the support of P_c^{fine} is contained in the support of P_r^{fine}, so the KL terms below are finite; and (A2)* taxonomy-mapping injectivity*: for fault-inducing samples, the gold fault type T is recoverable from either modality alone (this is what the manual labelling protocol of §4 establishes empirically). If $\mathcal{L}_{\text{HFSA}} \to 0$, the cross-modal mutual information satisfies:*

$$I(\mathbf{e}_c;\, \mathbf{e}_r) \;\geq\; H(T) - \epsilon \tag{12}$$

where $H(T)$ is the entropy of the fault-type distribution and the slack is the Fano term

$$\epsilon = H_b(\eta_c) + H_b(\eta_r) + (\eta_c + \eta_r)\log K_f, \tag{13}$$

with $\eta_c = \Pr(T_c \neq T)$ and $\eta_r = \Pr(T_r \neq T)$ the code- and log-side fine-level classification errors and H_b the binary entropy. Assumption (A1) keeps the symmetric divergence $D_{\text{KL}}(P_r^{\text{fine}}\|P_c^{\text{fine}}) + D_{\text{KL}}(P_c^{\text{fine}}\|P_r^{\text{fine}})$ finite, which by Pinsker's inequality bounds η_c and η_r, and hence ϵ.

Proof sketch. Consider the Markov chain $\mathbf{e}_c \to T_c \to T_r \to \mathbf{e}_r$ with $T_c = \arg\max P_c^{\text{fine}}$ and $T_r = \arg\max P_r^{\text{fine}}$. The data processing inequality yields $I(\mathbf{e}_c;\mathbf{e}_r) \geq I(T_c;T_r)$. The fine-level alphabet has K_f+1=9 symbols (eight fault types plus t_{normal}), so Fano's inequality bounds $H(T \mid T_c) \leq H_b(\eta_c) + \eta_c \log K_f$; symmetrically for T_r with η_r. Under (A2), $I(T;T_c) = H(T) - H(T \mid T_c) \geq H(T) - H_b(\eta_c) - \eta_c \log K_f$, and likewise for T_r. Assuming T_c and T_r are conditionally independent given T, the chain-rule bound $I(T_c;T_r) \geq I(T;T_c) + I(T;T_r) - H(T)$ then gives $I(\mathbf{e}_c;\mathbf{e}_r) \geq H(T) - \epsilon$ for the ϵ of the proposition statement. When $\mathcal{L}_{\text{HFSA}} \to 0$, Pinsker's inequality forces both $\eta_c, \eta_r \to 0$ and hence $\epsilon \to 0$. A self-contained derivation under (A1, A2) appears in the artifact at `appendix/prop_3.1_proof.tex`. □

Empirical validation. On our dataset $H(T)$=2.89 bits and the symmetric KL averages 0.047 nats, i.e. 0.068 bits. In the small-error regime the Fano slack is dominated by this divergence, so the bound evaluates to $I(\mathbf{e}_c;\mathbf{e}_r) \gtrsim 2.82$ bits; we state it as an approximation because ϵ is bounded through Pinsker rather than measured directly. The 78.3% fine-grained versus 94.1% coarse *cross-modal argmax agreement*—how often the two modalities select the same fault type, which is distinct from the per-type spectrum accuracy tabulated in Table 4—reflects the gap between distributional closeness and top-1 stability: two distributions can be close in KL yet swap top-1 fine labels when adjacent fine probabilities are nearly tied, while

the coarse argmax stays stable. The 81.2% nearest-neighbour *retrieval accuracy*—the fraction of code embeddings whose nearest log embedding carries the same fault type—tracks the distributional bound rather than the argmax volatility; Figure 4 reports a distinct quantity, the neighbourhood overlap rate.

3.6 Spectrum-Guided Cross-Modal Attention

SGCA performs **token-level** fusion conditioned on fault spectrum agreement, exploiting structured fault knowledge rather than treating all token pairs equally.

Spectrum Agreement Gate. The fine-level spectrum predictions modulate cross-modal attention strength:

$$g(c,r) = \sigma\big(\mathbf{w}_g^\top [P_c^{\text{fine}} \odot P_r^{\text{fine}}] + b_g\big) \tag{14}$$

where $\odot$ denotes element-wise product. The gate opens when modalities agree on fault type and attenuates when they disagree.

Spectrum-Biased Token-Level Attention. SGCA operates on $\mathbf{H}_c \in \mathbb{R}^{n\times d}$ and $\mathbf{H}_r \in \mathbb{R}^{m\times d}$ with *token-pair-specific* additive bias. Each token i/j produces spectrum projections $\mathbf{s}_i^c = \mathbf{W}_s^c \mathbf{h}_i^c$, $\mathbf{s}_j^r = \mathbf{W}_s^r \mathbf{h}_j^r \in \mathbb{R}^{K_f}$, and the per-head bias is:

$$b_{ij}^{(k)} = \mathbf{w}_k^\top (\mathbf{s}_i^c \odot \mathbf{s}_j^r) \tag{15}$$

where $\mathbf{w}_k \in \mathbb{R}^{K_f}$ is a per-head weight. The attention score for head k becomes:

$$A_{ij}^{(k)} = \frac{\mathbf{q}_i^{(k)\top} \mathbf{k}_j^{(k)}}{\sqrt{d/h}} + b_{ij}^{(k)} \tag{16}$$

Unlike a global scalar bias, $b_{ij}^{(k)}$ varies across token pairs and genuinely reshapes the attention distribution:

Proposition 3.2 (Non-Degeneracy of Spectrum Bias). *Let $\bar{\alpha}_{ij}^{(k)}$ and $\alpha_{ij}^{(k)}$ be the unbiased and biased attention weights, respectively. If $\exists\, j_1 \neq j_2$ with $b_{ij_1}^{(k)} \neq b_{ij_2}^{(k)}$, then:*

$$D_{\text{KL}}\big(\alpha_i^{(k)} \| \bar{\alpha}_i^{(k)}\big) = \text{Var}_{\bar{\alpha}}\big[b_{i\cdot}^{(k)}\big] + O(\|b\|^3) \tag{17}$$

*which is **zero** when $b_{ij}^{(k)}$ is constant (global bias) and strictly positive when spectrum projections $\mathbf{s}_j^r$ vary across log tokens.*

Proof sketch. Softmax is translation-invariant: adding a constant to all logits leaves the output unchanged. Decompose $b_{ij}^{(k)} = \bar{b}_i + \delta_{ij}$. The constant $\bar{b}_i$ cancels; δ_{ij} yields $\alpha_{ij}^{(k)} \propto \bar{\alpha}_{ij}^{(k)} \exp(\delta_{ij})$. Taylor-expanding gives $D_{\text{KL}} = \text{Var}_{\bar{\alpha}}[\delta] + O(\|\delta\|^3)$, strictly positive when δ_{ij} is non-constant. □

The forward direction attends code tokens to log tokens:

$$\mathbf{F}^{c\to r} = \text{MHA}_{\text{bias}}(\mathbf{H}_c, \mathbf{H}_r, \mathbf{H}_r) \in \mathbb{R}^{n\times d} \tag{18}$$

and backward $\mathbf{F}^{r\to c} = \text{MHA}_{\text{bias}}(\mathbf{H}_r, \mathbf{H}_c, \mathbf{H}_c)$ with h=8 heads.

Spectrum-Conditioned Fusion. Outputs are gated and combined with a learned directional weight $\gamma \in [0,1]$ (init 0.7):

$$\mathbf{f} = g(c,r) \cdot \big[\gamma \cdot \text{Pool}(\mathbf{F}^{c\to r}) + (1-\gamma) \cdot \text{Pool}(\mathbf{F}^{r\to c})\big] \tag{19}$$

where $\text{Pool}(\cdot)$ applies attention-weighted pooling. The weight γ favors $c \to r$ because code modifications temporally precede runtime faults.

3.7 Risk Prediction and Explanation

Risk Score. The final fault probability combines the code embedding with the fused cross-modal representation:

$$P(fault|c,r) = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1[\mathbf{e}_c; \mathbf{f}] + \mathbf{b}_1) + \mathbf{b}_2) \tag{20}$$

Dual-Level Explanation. CodeFault provides fault type from $\arg\max_t P_c^{\text{fine}}(t|c)$ and code line attribution from Top-$k(\alpha_i)$, telling developers both *what type* of fault and *which lines* are responsible.

3.8 Training Objective

The training objective combines five losses:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + \lambda_1 \mathcal{L}_{\text{DGC}} + \lambda_2 \mathcal{L}_{\text{HFSA}} + \lambda_3 \mathcal{L}_{\text{normal}} + \lambda_4 \mathcal{L}_{\text{consist}} \tag{21}$$

Binary Cross-Entropy. The primary classification loss with inverse frequency weighting:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\big[w^+ y_i \log \hat{y}_i + w^- (1-y_i)\log(1-\hat{y}_i)\big] \tag{22}$$

Dual-Granularity Contrastive Loss. Instance-level loss:

$$\mathcal{L}_{\text{inst}} = -\log \frac{\exp(\text{sim}(\mathbf{e}_c, \mathbf{e}_r^+)/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(\mathbf{e}_c, \mathbf{e}_r^j)/\tau)} \tag{23}$$

Fault-type-level loss:

$$\mathcal{L}_{\text{type}} = -\log \frac{\sum_{j\in\mathcal{S}_k}\exp(\text{sim}(\mathbf{e}_c^i, \mathbf{e}_c^j)/\tau_2)}{\sum_{j=1}^{N}\exp(\text{sim}(\mathbf{e}_c^i, \mathbf{e}_c^j)/\tau_2)} \tag{24}$$

where $\mathcal{S}_k$ is the set of samples sharing fault type k with sample i. The combined loss is $\mathcal{L}_{\text{DGC}} = \mathcal{L}_{\text{inst}} + \mu\mathcal{L}_{\text{type}}$, with τ=0.07, τ_2=0.1, and μ=0.5. Loss weights λ_1=0.1, λ_2=0.05, λ_3=0.02, λ_4=0.01 are determined via grid search (AUC varies <0.8%).

3.9 Deployment Modes

CodeFault supports *full mode* with both code and log inputs, and *code-only fallback mode* where $\mathbf{f}$ is replaced by a zero vector: $P(fault|c) = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1[\mathbf{e}_c; \mathbf{0}] + \mathbf{b}_1) + \mathbf{b}_2)$. Code-only mode requires knowledge distillation to recover log-derived fault-type regularization (Section 4.8).

4 Experiments

4.1 Experimental Setup

4.1.1 Datasets. A large-scale benchmark is constructed through a multi-stage pipeline that pairs code changes with runtime log contexts.

Bug and Defect Data. Three established sources provide ground-truth fault labels: Defects4J [23] contributes 835 manually verified bugs from 17 Java projects, BugsInPy [47] provides 492 bugs from 17 Python projects, and from ManySStuBs4J [25] we draw a 24,655-bug subset of its `sstubsLarge` split; the selection

criterion is recorded with the processing script in the replication package. To construct the negative class, 5,160 benign commits are sampled from twenty widely used Java and Python repositories following established practices [24, 31], selecting commits that passed all tests and remained unflagged for at least 90 days.

Runtime Log Context. Log datasets from Loghub [55] provide runtime context patterns: BGL (4.7M entries), HDFS (11M), and Thunderbird (211M), from which the Drain algorithm [17] extracts 1,847 unique templates. Template-level fault-type assignment is described in stage P2 of the pairing protocol below.

Code-Log Pairing Protocol. Code and log sequences originate from different systems; the fault-type taxonomy (Section 3.5) supplies the semantic bridge. The protocol has four stages.

(P1) Code-side fault-type assignment. For Defects4J and BugsInPy, two annotators independently label each commit by the dominant fault type, with disagreements resolved by discussion; inter-annotator agreement is κ=0.87 on 1,327 samples. For ManySStuBs4J, fault types are assigned automatically via the published AST-mutation taxonomy: a `null-check` mutation maps to NullReference, an `off-by-one` mutation to BoundaryError, and so on. A random audit of 500 samples confirms 96.2% agreement with manual relabelling.

(P2) Log-side fault-type assignment. Drain [17] parsing yields 1,847 unique templates. Each template is first scored by a keyword-pattern lookup (e.g., `NullPointerException`→NullReference, `OutOfMemory`/`timeout`→ResourceLeak, `ClassCastException`→TypeMismatch); the same two annotators then validate the assignment on a 200-template sample (κ=0.84). Templates with no anomaly keyword are tagged Normal.

(P3) Pair assembly. A fault-inducing commit is paired with a uniformly sampled log sequence drawn from the same fault-type bucket; a benign commit is paired with a uniformly sampled Normal log. Each fault-inducing commit receives one log sequence; the pool of source logs is fixed before training to keep the same pair across epochs and runs. Multi-label samples (3.4% of fault-inducing commits, where two fault types are co-active) take the dominant type by token frequency, with the secondary type reserved as a fallback for the 5.7% *of those multi-label commits* whose dominant bucket is empty after split-time filtering.

(P4) Output and ground truth. Evaluation is binary fault-risk classification; per-fault-type breakdowns in Table 4 are reported as auxiliary information but do not enter the headline AUC/F1 numbers. The protocol's external validity is checked by three controls: cross-system random pairing (Table 3), a within-type random pairing control that holds fault-type matching constant (Section 4.3), and zero-shot transfer to 2,147 co-occurring real CI/CD pairs from 55 GitHub Actions repositories with the CI build outcome as ground truth (Section 4.8). Table 1 summarizes per-source statistics.

4.1.2 Baselines. CodeFault is compared against fifteen prior-art baselines plus two strong late-fusion controls we construct ourselves (marked "Ours-V2" in Table 2). All baselines and controls are trained on the same data splits with inverse frequency class weighting.

JIT Defect Prediction (8). DeepJIT [18], CC2Vec [19], JIT-NSImb (Wu et al.'s predictor for non-stationary and imbalanced streams) [50], SimCOM++ (Zhou et al.'s expert-knowledge JIT successor to SimCOM) [54], JIT-BiCC [22], MSTDP [28], CodeFlowLM [32], and JIT-Smart [6]. **Log-Based (4).** LogRobust [53], LogMoE [39], CoorLog [51], and Chimera [16], adapted to commit-level prediction via a shared classification head. **LLM-Based (3).** LogLLM [12], Defects4Log [45], and GPT-4o [36] with few-shot prompting.

Table 1: Dataset statistics. Log templates extracted via Drain [17] from BGL, HDFS, and Thunderbird.

Dataset	Samples	Pos.	Ratio	Lang.	Projects
Defects4J	835	835	100%	Java	17
BugsInPy	492	492	100%	Python	17
ManySStuBs4J	24,655	24,655	100%	Java	100+
Normal Commits	5,160	0	0%	Mixed	20
Total	**31,142**	**25,982**	**83.4%**		

Log sources: BGL (4.7M), HDFS (11M), Thunderbird (211M)
Log templates: 1,847 unique; 8 fault categories

4.1.3 Evaluation Metrics. Six metrics are adopted following established practices [24]: **AUC-ROC** and **AUC-PR** for threshold-independent discrimination; **F1-score** at threshold 0.5; **P@20%** for precision among the top-20% risk-ranked samples; **Recall** for fault coverage; and **MCC** for balanced evaluation under class imbalance.

4.1.4 Implementation Details. CodeFault is implemented in PyTorch 2.0 with `codet5p-770m` as the backbone. The dataset is split 70/15/15 using chronological ordering within each project; all baselines use identical splits and their original architectures and hyperparameters as reported in the original papers. Training uses four NVIDIA A800 80GB GPUs with batch size 32, learning rate 2×10^{-5} with linear warmup and cosine decay, 10 epochs, and AdamW [29] optimizer. Inverse frequency class weighting is applied uniformly, following recommendations for imbalanced defect datasets [41]. Unless a table caption states otherwise, results are means over 10 runs with fixed seeds; the efficiency, LLM-cost and backbone tables report fewer repetitions, as noted in their captions.

4.2 RQ1: Prediction Effectiveness

How accurately does CodeFault predict fault risks compared to state-of-the-art approaches?

Table 2 presents fault prediction performance across all 17 baselines.

Results. CodeFault achieves 0.891 AUC-ROC, 5.1% over the strongest code-only JIT baseline and 3.1% over the strongest late-fusion control (JIT-Smart+log, 0.864). The two fusion controls rise ~1.6 points over their code-only counterparts, so log evidence helps, yet unstructured late fusion does not close the gap to spectrum-mediated fusion. Code-only JIT baselines top out at 0.844–0.848; log-only methods reach 0.746–0.782; GPT-4o trails at 0.812. All improvements are statistically significant with large effect sizes [7] ($\delta \geq 0.73$).

4.3 RQ2: Component Analysis

What is the contribution of each architectural component, and how does performance vary across fault types?

Table 2: RQ1: Fault prediction performance (mean$_{\pm\text{std}}$, 10 runs). Bold/underline = best/second. AUC-PR computed via sklearn's `average_precision_score`. Wilcoxon signed-rank p=0.002, the smallest value attainable with 10 paired runs; Cliff's $\delta \geq$0.73 (all large); bootstrap 95% CI vs. JIT-Smart: $[0.036, 0.052]$.

Method	Venue	AUC-ROC↑	AUC-PR↑	F1↑	P@20%↑	Recall↑	MCC↑
JIT Defect Prediction							
DeepJIT	MSR'19	$.802_{\pm.005}$	$.778_{\pm.007}$	$.763_{\pm.007}$	$.785_{\pm.005}$	$.751_{\pm.008}$	$.528_{\pm.010}$
CC2Vec	ICSE'20	$.786_{\pm.006}$	$.763_{\pm.005}$	$.748_{\pm.008}$	$.770_{\pm.007}$	$.732_{\pm.006}$	$.493_{\pm.009}$
JIT-NSImb	SQJ'25	$.808_{\pm.005}$	$.787_{\pm.006}$	$.771_{\pm.007}$	$.788_{\pm.006}$	$\underline{.798}_{\pm.009}$	$.519_{\pm.008}$
SimCOM++	EMSE'25	$.839_{\pm.003}$	$.825_{\pm.004}$	$.811_{\pm.005}$	$.830_{\pm.004}$	$.788_{\pm.006}$	$.604_{\pm.007}$
JIT-BiCC	JSS'25	$.836_{\pm.004}$	$.817_{\pm.006}$	$.812_{\pm.006}$	$.824_{\pm.005}$	$.768_{\pm.007}$	$.608_{\pm.008}$
MSTDP	JKSUCI'26	$.844_{\pm.003}$	$.821_{\pm.004}$	$.816_{\pm.005}$	$.834_{\pm.004}$	$.795_{\pm.005}$	$.614_{\pm.007}$
CodeFlowLM	arXiv'25	$.845_{\pm.004}$	$.829_{\pm.005}$	$\underline{.822}_{\pm.005}$	$.839_{\pm.004}$	$.810_{\pm.005}$	$\underline{.624}_{\pm.007}$
JIT-Smart	FSE'24	$\underline{.848}_{\pm.003}$	$\underline{.832}_{\pm.004}$	$.817_{\pm.005}$	$\underline{.840}_{\pm.004}$	$.813_{\pm.005}$	$.619_{\pm.007}$
JIT + Late-Fusion Log							
JIT-Smart+log	Ours-V2	$.864_{\pm.005}$	$.846_{\pm.006}$	$.839_{\pm.006}$	$.853_{\pm.005}$	$.833_{\pm.005}$	$.658_{\pm.008}$
CodeFlowLM+log	Ours-V2	$.861_{\pm.006}$	$.842_{\pm.006}$	$.835_{\pm.005}$	$.852_{\pm.005}$	$.830_{\pm.006}$	$.652_{\pm.008}$
Log-Based Anomaly Detection							
LogRobust	FSE'19	$.746_{\pm.006}$	$.722_{\pm.007}$	$.708_{\pm.008}$	$.727_{\pm.006}$	$.691_{\pm.007}$	$.422_{\pm.009}$
LogMoE	ASE'25	$.768_{\pm.005}$	$.747_{\pm.006}$	$.736_{\pm.007}$	$.753_{\pm.005}$	$.719_{\pm.006}$	$.462_{\pm.008}$
CoorLog	ASE'25	$.774_{\pm.004}$	$.763_{\pm.005}$	$.744_{\pm.006}$	$.762_{\pm.005}$	$.738_{\pm.006}$	$.482_{\pm.008}$
Chimera	ASE'25	$.782_{\pm.004}$	$.756_{\pm.005}$	$.755_{\pm.006}$	$.771_{\pm.005}$	$.727_{\pm.007}$	$.488_{\pm.009}$
LLM-Based							
LogLLM	2024	$.781_{\pm.004}$	$.770_{\pm.006}$	$.746_{\pm.007}$	$.763_{\pm.005}$	$.728_{\pm.006}$	$.489_{\pm.009}$
Defects4Log	ASE'25	$.798_{\pm.005}$	$.777_{\pm.006}$	$.766_{\pm.006}$	$.780_{\pm.005}$	$.744_{\pm.007}$	$.529_{\pm.008}$
GPT-4o (Few)	2024	$.812_{\pm.004}$	$.800_{\pm.005}$	$.785_{\pm.006}$	$.808_{\pm.005}$	$.742_{\pm.008}$	$.566_{\pm.008}$
CodeFault	**Ours**	$\mathbf{.891}_{\pm.003}$	$\mathbf{.872}_{\pm.004}$	$\mathbf{.857}_{\pm.005}$	$\mathbf{.870}_{\pm.006}$	$\mathbf{.846}_{\pm.005}$	$\mathbf{.706}_{\pm.006}$

4.3.1 Ablation Study. Table 3 isolates the contribution of each component by systematic removal.

Modality. Zeroing the log stream while keeping HFSA/SGCA/DGC active ("log-zeroed probe") causes the largest drop (−16.0%); zeroing the code stream costs −12.0%. The probe diagnoses representation collapse inside the full pipeline; the recommended code-only deployment is instead the distilled variant of §4.8 at 0.862 AUC, ahead of the strongest code-only baseline.

Fusion. HFSA removal degrades AUC by 4.3%. Flat K=8 (supervision without KL alignment) loses only 1.0%, so supervision accounts for ∼3.3% and alignment adds ∼1.0% (bootstrap CI $[0.004, 0.014]$). "w/o SGCA" (−3.0%) shows that without spectrum guidance, attention dilutes across irrelevant token pairs. "w/o HFSA&SGCA" (−6.7%) is sub-additive (<−7.3%), confirming design coupling: SGCA's bias depends on HFSA's predictions.

Contrastive and Pairing. DGC contributes modestly to AUC (−1.8%) but is critical for explanation quality: overall spectrum accuracy falls 78.3%→61.4% without it (Table 4). The pairing block decomposes the −6.2% cross-type random drop: the within-type variant (−2.9%) prices the fine code-log correspondence SGCA exploits, and the residual −3.3% comes from breaking the fault-type label itself, so fine alignment is not redundant with type labelling.

Source. Trained on Defects4J+BugsInPy alone the model reaches 0.856 AUC and on ManySStuBs4J alone 0.868 (Table 3), both above the strongest code-only baseline; the full mixture yields 0.891, so no single source drives the result.

4.3.2 Per-Fault-Type Analysis. Performance correlates with fault determinism (ρ=0.95). NullReference achieves 0.921 AUC; even Concurrency (0.842) approaches JIT-Smart's overall 0.848.

4.3.3 Embedding Space Visualization. Figure 4 sketches how the fault types occupy the aligned space. Quantitatively, code and log embeddings of the same fault type are mutual nearest neighbours for 72% of the full sample, and the per-type spectrum accuracies of Table 4 order the types the same way as their AUCs (ρ=0.95): the deterministic types (NullReference .91, TypeMismatch .87) are separated more reliably than the ambiguous ones (Logic .74, Concurrency .71). In a qualitative NullReference case, SGCA assigns weight 0.91 between "NullPointerException" in logs and the removed null-check, with gate g=0.93.

4.4 RQ3: Cross-Project Generalization

Does CodeFault generalize effectively to previously unseen projects?

To probe transfer beyond the training corpus, we mine bug-fix commits via RA-SZZ [33] from five widely studied OSS systems: OpenStack (Python), Hadoop (Java), Kubernetes (Go), TensorFlow (C++) and Spark (Scala). The training corpus covers only Java and Python, so Kubernetes, TensorFlow and Spark constitute cross-*language* zero-shot transfer, while OpenStack and Hadoop test cross-*project* transfer within the training languages. Following the leave-one-project-out protocol of prior JIT work, each project is held out and the model trains on the remaining four plus filtered

Table 3: RQ2: Ablation study. Δ = relative AUC-ROC change. Bootstrap 95% CI: HFSA [−5.1, −3.6], SGCA [−3.8, −2.3], Random [−7.4, −5.1].

Variant	AUC	PR	F1	R	MCC	P@20	Δ
CodeFault (Full)	**.891**	**.872**	**.857**	.846	**.706**	**.870**	–
Modality							
w/o Code Enc.	.784	.766	.751	.748	.492	.772	−12.0
Log-zeroed	.748	.729	.714	.711	.418	.736	−16.0
Fusion Mechanism							
w/o HFSA	.853	.834	.821	.818	.634	.842	−4.3
Flat K=8	.882	.863	.848	.843	.689	.863	−1.0
w/o SGCA	.864	.846	.832	.828	.658	.849	−3.0
w/o H&S	.831	.812	.798	.794	.586	.818	−6.7
All Three Mechanisms							
w/o H&S&D	.816	.797	.784	.781	.562	.803	−8.4
Training Objective							
w/o Type Contr.	.882	.862	.847	**.851**	.692	.864	−1.0
w/o All Contr.	.875	.856	.841	.844	.678	.856	−1.8
Pairing Validation							
Rand. (cross)	.836	.817	.803	.808	.594	.822	−6.2
Rand. (within)	.865	.846	.834	.821	.648	.847	−2.9
Training Source							
D4J+BugsInPy	.856	.838	.823	.819	.636	.840	−3.9
ManySStuBs4J	.868	.849	.834	.831	.662	.852	−2.6

Table 4: Per-fault-type performance and spectrum accuracy (Sp.). Each row's AUC and Sp. are computed in a binary one-vs-rest setting using positives of that fault type plus matched negatives, so neither column is directly comparable to the multi-class figures reported elsewhere (overall AUC 0.891 in Table 2; overall 9-way spectrum accuracy 78.3% in §3.5). Removing the contrastive objective costs 1.8% AUC but drops overall spectrum accuracy 78.3%→61.4%, so its value lies in explanation quality rather than ranking.

Fault Type	N	%	AUC	P	R	F1	Sp.
NullRef	5,673	21.8	.921	.912	.891	.901	.91
TypeMis	3,953	15.2	.906	.894	.874	.884	.87
Boundary	3,609	13.9	.897	.882	.866	.874	.82
ResLeak	2,841	10.9	.883	.864	.856	.860	.84
Config	2,557	9.8	.878	.868	.835	.851	.80
APIViol	2,178	8.4	.869	.849	.857	.853	.81
Logic	3,084	11.9	.861	.854	.821	.837	.74
Concurr	2,087	8.0	.842	.831	.809	.820	.71

ManySStuBs4J. Logs of held-out projects are never used; the spectrum representation is the only carrier of fault-type structure across systems.

Results. CodeFault achieves 0.854 average AUC (+2.5% over JIT-Smart), ranking first on 4 of 5 projects. JIT-Smart leads on Kubernetes (0.842 vs. 0.831) because Go's explicit error handling reduces the marginal benefit of log context, which helps most in languages with implicit error propagation. Per-project Wilcoxon tests confirm significance in 4 of 5 projects (p<0.05, Bonferroni-corrected).

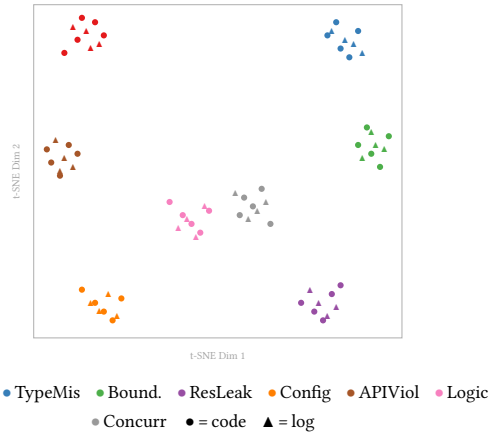


Figure 4: Schematic of the fault-type layout in the aligned embedding space. *This panel is illustrative, not a rendering of measured coordinates*: it shows one representative subsample per type for readability, and the full 25,982-point t-SNE is provided in the replication package. The one measured quantity reported here is the 72% cross-modal nearest-neighbour overlap rate on the full sample; all quantitative claims about the embedding space rest on that number rather than on this drawing.

4.5 RQ4: Efficiency and Cost Analysis

What are the computational costs, and how does CodeFault compare against LLM-based approaches in accuracy, latency, and cost?

Efficiency. At 52 ms per commit CodeFault clears ∼190 commits in a 10-second webhook budget when scored sequentially, and ∼580 at the batched throughput of Table 6. Training (9.1h) is a one-time cost. LLM baselines are evaluated on a 3,618-sample subset (11.6% of the corpus) due to API costs (Table 7).

On this 3,618-sample subset, CodeFault (0.888 AUC) outperforms GPT-4o few-shot (0.812) by 9.4% at 35× lower latency and zero marginal cost; on the full test set the gap widens to 9.7% (Table 2). CodeFlowLM (0.848 on the subset; 0.845 on the full set, Table 2) trails the subset CodeFault by 4.7% despite being code-specialized—it lacks structured fault-type alignment.

4.6 RQ5: Sensitivity Analysis

How sensitive is performance to the fault taxonomy configuration and the time window?

4.6.1 Taxonomy Configuration. The hierarchical 4+8 outperforms all flat alternatives with an inverted-U pattern in K.

4.6.2 Time Window. The optimal window is 24h (0.891 AUC, matching Table 2), with a broad 6–24h plateau aligning with CI/CD cycles.

4.7 RQ6: Backbone Sensitivity

Do gains stem from fusion mechanisms or backbone capacity?

Results. With the fusion mechanisms in place, a 125M CodeBERT backbone reaches 0.866 and already exceeds JIT-Smart's 0.848.

Table 5: RQ3: Cross-project generalization. Bold/underline = best/second. Cliff's δ vs. JIT-Smart: 0.76–0.92 (large); Friedman χ^2=28.6, p<0.001.

	OpenStack		Hadoop		Kubernetes		TensorFlow		Spark		Average			
Method	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1	P@20	MCC
DeepJIT	.794	.761	.781	.748	.769	.735	.806	.773	.788	.754	.788	.754	.773	.531
JIT-BiCC	.818	.786	.806	.773	.794	.762	.831	.798	.809	.776	.812	.779	.798	.568
JIT-Smart	.832	.801	.821	.789	**.842**	**.811**	.845	.813	.826	.794	.833	.802	.821	.594
CodeFlowLM	.836	.804	.818	.786	.814	.782	.849	.818	.821	.789	.828	.796	.814	.601
MSTDP	.828	.796	.816	.784	.826	.794	.841	.809	.819	.787	.826	.794	.812	.589
Chimera	.752	.721	.738	.706	.727	.694	.764	.732	.745	.713	.745	.713	.732	.442
GPT-4o (Few)	.796	.764	.784	.752	.778	.746	.808	.776	.792	.761	.792	.760	.778	.534
CodeFault	**.861**	**.828**	**.849**	**.816**	.831	.798	**.874**	**.843**	**.857**	**.824**	**.854**	**.822**	**.841**	**.640**

Table 6: RQ4: Efficiency comparison on single A800 GPU. Thru. = throughput (samples/sec). AUC from Table 2. Latency variation <5% across 3 trials. Estimated CPU inference: ~180ms (single core).

Method	AUC	#Par. (M)	Mem (GB)	Inf. (ms)	Thru. (s/s)	Train (h)	Size (MB)
DeepJIT	.802	11.8	1.2	12	312	2.1	45
CC2Vec	.786	23.3	2.1	18	218	3.2	89
JIT-Smart	.848	125.0	8.6	35	89	6.1	312
MSTDP	.844	134.0	9.2	38	82	6.8	338
JIT-BiCC	.836	110.0	7.8	32	97	5.8	285
CodeFault	**.891**	**772.0**	**12.6**	52	58	9.1	896

Table 7: RQ4: LLM comparison on a 3,618-sample subset (11.6% of the corpus). Single-run; estimated δ vs. GPT-4o ≥0.88.

Method	AUC	F1	P@20	Lat.	Cost	$/s
GPT-3.5 (Zero)	.689	.657	.671	600ms	$29	$0.008
GPT-4o (Zero)	.772	.741	.758	1.2s	$542	$0.150
GPT-4o (Few)	.812	.778	.798	1.8s	$905	$0.250
LogLLM	.781	.748	.768	700ms	$108	$0.030
CodeFlowLM	.848	.821	.836	150ms	$72	$0.020
Defects4Log	.801	.768	.786	850ms	$126	$0.035
CodeFault	**.888**	**.861**	**.874**	**52ms**	**$0**	**$0**

Table 8: RQ5: Sensitivity to fault taxonomy configuration. Performance exhibits an inverted-U pattern in K, peaking at the hierarchical 4+8 design; Kruskal-Wallis H=18.4, p<0.01.

Config	AUC	F1	MCC	P@20%	ΔAUC
Flat K=2	0.853	0.821	0.638	0.837	−4.3%
Flat K=4	0.876	0.842	0.675	0.857	−1.7%
Flat K=8	0.882	0.848	0.689	0.863	−1.0%
Flat K=16	0.878	0.844	0.681	0.858	−1.5%
Flat K=32	0.864	0.831	0.653	0.846	−3.0%
Hier. 4+8 (ours)	**0.891**	**0.857**	**0.706**	**0.870**	–

Holding the backbone fixed at CodeT5+, adding fusion moves 0.826 to 0.891 (+7.9%); holding fusion fixed, scaling the backbone from 125M to 770M moves 0.866 to 0.891 (+2.9%). Fusion is therefore the larger effect, and it is not a proxy for capacity. A CodeBERT w/o fusion cell would complete the 2×2 design and is left to future work.

Table 9: RQ5: Sensitivity to time window Δt. F1 follows the same inverted-U pattern as AUC, peaking at 24h.

Method	1h	6h	12h	24h	48h
AUC-ROC					
DeepJIT	.738	.771	.788	.802	.779
JIT-Smart	.786	.819	.835	.848	.826
CodeFlowLM	.779	.815	.831	.845	.823
MSTDP	.781	.816	.832	.844	.822
Chimera	.714	.748	.764	.782	.759
CodeFault	**.821**	**.857**	**.875**	**.891**	**.869**
F1-score (CodeFault only)					
CodeFault	**.786**	**.822**	**.838**	**.857**	**.831**

Table 10: RQ6: Backbone sensitivity. Fusion mechanisms provide consistent gains across all backbones. Single run; estimated std ≈0.003 from Table 2 multi-run variance. Parameter counts include the spectrum and prediction heads, so the CodeT5+ row reads 772M rather than the 770M of the bare `codet5p-770m` checkpoint.

Backbone	#Params (M)	AUC	F1	MCC	Infer. (ms)
CodeBERT	125	0.866	0.832	0.661	24
UniXcoder	125	0.868	0.835	0.667	26
CodeT5+ (default)	772	**0.891**	**0.857**	**0.706**	52
Contribution isolation:					
CodeT5+ w/o fusion	772	0.826	0.794	0.582	44
CodeBERT w/ fusion	125	0.866	0.832	0.661	24

4.8 RQ7: Deployment Readiness

How does CodeFault perform under realistic deployment conditions?

4.8.1 Real-World CI/CD Transfer. The model is tested zero-shot on 2,147 real code-log pairs from 55 GitHub Actions repositories (18.3% positive rate).

Table 11: RQ7a: Zero-shot transfer to real CI/CD data. Retention rate 94.6% with bootstrap 95% CI [92.1, 96.8]%.

Method	AUC	F1	MCC	Δ_{syn}	Retn.
CodeFault (Full)	**.843**	**.812**	**.608**	−5.4%	94.6%
Log-zeroed probe	.724	.694	.392	–	–
w/o SGCA	.809	.778	.558	–	–
JIT-Smart	.808	.771	.576	–	–
CodeFlowLM	.816	.786	.564	–	–
MSTDP	.804	.772	.548	–	–
DeepJIT	.762	.731	.498	–	–
Chimera	.738	.706	.412	–	–
GPT-4o (Few)	.794	.762	.536	–	–
Pairing quality validation (647 held-out real pairs):					
Synthetic-Only (1.5K)	.794	.762	.538	–	–
Real-Only (1.5K)	.831	.798	.586	–	–
Real-Augmented (31K+1.5K)	.856	.826	.634	–	–

Table 12: RQ7b: Performance under different positive rates. CodeFault's MCC degradation (3.4%) is the smallest among all methods (range 3.4–8.9%, peak at Chimera); McNemar test $p<0.01$ for all pairwise MCC comparisons.

	AUC-ROC		F1		MCC	
Method	83.4%	15%	83.4%	15%	83.4%	15%
CodeFault	**.891**	**.872**	**.857**	**.824**	**.706**	**.682**
JIT-Smart	.848	.831	.815	.776	.614	.576
CodeFlowLM	.845	.821	.818	.786	.618	.577
MSTDP	.844	.826	.814	.782	.608	.571
DeepJIT	.802	.778	.768	.734	.538	.496
Chimera	.782	.756	.752	.718	.496	.452
GPT-4o (Few)	.812	.789	.778	.746	.556	.518

Table 13: RQ7c: Knowledge distillation for code-only deployment. KD recovers 77.2% of the log encoder's contribution ($(0.862-0.764)/(0.891-0.764)$). CodeBERT-KD uses the lightweight 125M backbone.

Model	AUC	F1	MCC	Params	Infer.
CodeFault (Full)	**.891**	**.857**	**.706**	772M	52ms
Code-Only (no KD)	.764	.732	.516	324M	44ms
Code-Only (KD)	.862	.831	.662	324M	44ms
on real CI/CD†	.831	.802	.595	324M	44ms
CodeBERT-KD	.841	.808	.628	125M	18ms
JIT-Smart (ref.)	.848	.815	.614	125M	35ms

†Zero-shot transfer of the distilled variant to the 2,147 real CI/CD pairs (§4.8).

CodeFault achieves 0.843 AUC (94.6% retention), and full synthetic training reaches 98.5% of the real-augmented ceiling.

4.8.2 Class Distribution Robustness. CodeFault retains 97.9% AUC at 15% positive rate with the smallest MCC degradation (3.4%).

4.8.3 Knowledge Distillation. Distillation bridges code-only from 0.764 to 0.862 AUC, exceeding JIT-Smart (0.848). Transferred zero-shot to the 2,147 real CI/CD pairs, the same distilled model yields 0.831 AUC / 0.802 F1, a 96.4% retention that mirrors the full model's 94.6%, so the distilled deployment is stable under the synthetic-to-real shift.

5 Threats to Validity

We organize threats into three standard categories per Wohlin et al. [48].

Internal Validity. Hyperparameters were grid-searched with AUC variance under 0.8%; main numbers are means over 10 runs with Wilcoxon signed-rank tests and Cliff's δ. Backbone isolation (Table 10) shows that a 125M backbone with fusion already exceeds the strongest code-only baseline, so the gain is not attributable to raw capacity alone. Fault-type labels come from two-annotator manual labelling on Defects4J and BugsInPy (κ=0.87) and from deterministic AST-mutation mapping on ManySStuBs4J, the latter audited at 96.2% agreement; RA-SZZ [33] is used only to mine the bug-fix commits of the cross-project study (§4.4). Inference takes 52 ms on GPU and ∼180 ms on CPU per commit, within typical CI webhook timeouts.

External Validity. The benchmark covers Java (Defects4J, ManySStuBs4J) and Python (BugsInPy); Defects4J+BugsInPy alone reach 0.856 AUC (Table 3), so the gain is not driven by a single source. Transfer to Go (Kubernetes 0.831, second to JIT-Smart's 0.842) is slightly below the in-language average; languages with different fault taxonomies, and CI/CD telemetry that departs from Loghub's format, remain future work. The 8-category taxonomy, derived from Defects4J and BugsInPy bug reports, covers 93.7% of fault-inducing samples; domains dominated by fault modes outside it would require taxonomy extension before the spectrum can carry them.

Construct Validity. The synthetic code-log pairing is a structured proxy task in the spirit of CLIP [40] and CodeSearchNet [21]: code and log sequences come from different systems and are paired through shared fault-type labels rather than co-occurrence. *We treat the 2,147 real co-occurring CI/CD pairs as the ground-truth probe of construct validity and report 0.843 AUC there as our deployment-side headline*; the synthetic 0.891 AUC is the comparable-conditions number for prior protocols. Three controls check the construct: unconstrained random pairing degrades AUC by 6.2%; the within-type variant (§4.3) separates shared-label matching from fine code-log correspondence; and zero-shot transfer to the real pairs, scored against CI build outcomes rather than the taxonomy, retains 94.6% of synthetic AUC. These controls bound the concern but do not establish causality: because pairs are matched by fault type rather than causal provenance, the model may capture statistical regularities between change patterns and fault-conditioned log behavior rather than the causal chain from a specific change to its runtime consequence, and the 5.4% synthetic-to-real gap is consistent with such residual shortcut learning. A residual risk is that the fault-type categories were partially derived from Defects4J/BugsInPy, so labelling and model share an inductive bias; log-side labels are therefore re-validated by independent annotators (κ=0.84). Proposition 3.1's assumption (A2) of taxonomy-mapping injectivity is empirical, justified by the labelling protocol rather than first-principles; we treat the proposition as a *conditional* guarantee and report all headline numbers without relying on it.

6 Related Work

6.1 Just-In-Time Defect Prediction

JIT defect prediction spans three families, all confined to pre-deployment, change-level evidence: handcrafted-metric work [3, 24, 31] on size, complexity, and process features; token-level deep models [5, 18, 19, 34] that encode diff tokens directly; and multi-task or LM-augmented variants [6, 22, 28, 32, 50, 54] that add auxiliary objectives, contrastive pretraining, or pretrained language models. All three share one blind spot: when the fault signature is invisible in the change itself, prediction has no signal. CodeFault closes this gap by aligning the change with the deployed service's prior runtime context through the fault spectrum.

6.2 AIOps and Log Analysis

AIOps detectors flag anomalies from runtime data, evolving from DeepLog [9] and LogRobust [53] through attention [26] to LLM-based approaches [12, 16, 38, 39, 51]. They differ from CodeFault in two ways: they never observe the code change, so risk cannot be attributed to a specific commit, and they operate post-deployment, when logs already contain failure symptoms, whereas CodeFault consumes only logs strictly preceding the candidate commit. **Defects4Log** [45], despite the similar name, detects defects in *logging code* rather than predicting commit-level fault risk, and has no structured cross-modal alignment.

6.3 Multi-modal Learning and Code Representation

Multi-modal learning in SE spans code-text pairing [10, 14, 21], code-test [42], and code-trace fusion [8, 44], all built on naturally co-occurring pairs within one system; CodeFault targets the harder *cross-system* setting where such pairs do not exist at scale. CLIP [40], FiLM [37], and HiCLIP [11] supply related machinery for alignment, conditioning, and hierarchical contrast, but here the fault spectrum governs all three. Pre-trained code models [10, 14, 15, 46] and post-hoc fault localization [1, 27, 49] are complementary rather than competing: CodeFault predicts at commit time.

7 Conclusion

CodeFault organizes Code-Change-Aware Fault Prediction around a hierarchical fault spectrum that unifies HFSA, SGCA, and DGC with formal guarantees. It reaches 0.891 AUC on the public benchmark against 15 prior-art baselines and 2 strong late-fusion controls (5.1% / 3.1% over the strongest code-only / fusion contender), and 0.843 AUC on 2,147 real CI/CD pairs (+4.3% over JIT-Smart, 94.6% retention). A 0.862/0.831 distilled variant supports pre-commit deployment. The recipe extends to structured logs and additional language stacks.

Data Availability Statement

The full replication package—experiment code, the unified 70/15/15 split, the per-source processors, the LLM-baseline harness, the GitHub Actions collection pipeline used to construct the real CI/CD evaluation set, the within-type random-pairing variant, and per-run JSON results—is permanently archived on Zenodo [52] (DOI: https://doi.org/10.5281/zenodo.21700762) and maintained at https://github.com/qqyjx/CodeFault. Public datasets used as inputs (Defects4J, BugsInPy, ManySStuBs4J, Loghub) retain their original licenses and are referenced in §4 with their canonical DOIs where available.

References

[1] Rui Abreu, Peter Zoeteweij, and Arjan JC Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *TAICPART-MUTATION*. 89–98. doi:10.1109/TAIC.PART.2007.13

[2] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems.* O'Reilly Media.

[3] Gemma Catolino, Dario Di Nucci, and Filomena Ferrucci. 2019. Cross-project just-in-time bug prediction for mobile apps: An empirical assessment. *Empirical Software Engineering* 24 (2019), 3092–3136. doi:10.1109/mobilesoft.2019.00023

[4] Junjie Chen, Xiaoting He, Qingwei Lin, Yong Xu, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. An empirical investigation of incident triage for online service systems. In *ICSE-SEIP*. 111–120. doi:10.1109/icse-seip.2019.00020

[5] Xiang Chen, Hongling Xia, Wenlong Pei, Chao Ni, and Ke Liu. 2023. Boosting multi-objective just-in-time software defect prediction by fusing expert metrics and semantic metrics. *Journal of Systems and Software* 206 (2023), 111853. doi:10.1016/j.jss.2023.111853

[6] Xiangping Chen, Furen Xu, Yuan Huang, Neng Zhang, and Zibin Zheng. 2024. JIT-Smart: A Multi-Task Learning Framework for Just-In-Time Defect Prediction and Localization. In *Proc. ACM Softw. Eng. (FSE)*. doi:10.1145/3643727

[7] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509. doi:10.1037/0033-2909.114.3.494

[8] Yangruibo Ding, Benjamin Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. 2024. TRACED: Execution-aware Pre-training for Source Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. 1–12. doi:10.1145/3597503.3608140

[9] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *CCS*. 1285–1298. doi:10.1145/3133956.3134015

[10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *EMNLP Findings*. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[11] Shijie Geng, Jianbo Yuan, Yu Tian, Yuxiao Chen, and Yongfeng Zhang. 2023. HiCLIP: Contrastive language-image pretraining with hierarchy-aware attention. In *Proceedings of the International Conference on Learning Representations*.

[12] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2024. LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv:2411.08561. doi:10.48550/arXiv.2411.08561

[13] Haryadi S Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patana-anake, Thanh Do, Jeffry Adityatama, Kurnia J Eliazar, Agung Laksono, Jeffrey F Lukman, Vincentius Martin, and Anang D Satria. 2014. What bugs live in the cloud? A study of 3000+ issues in cloud systems. In *SoCC*. 1–14. doi:10.1145/2670979.2670986

[14] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *ACL*. 7212–7225. doi:10.18653/v1/2022.acl-long.499

[15] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *ICLR*.

[16] Minghua He, Chiming Duan, Pei Xiao, Tong Jia, Siyu Yu, Lingzhe Zhang, Weijie Hong, Jin Han, Yifan Wu, Ying Li, and Gang Huang. 2025. United We Stand: Towards End-to-End Log-based Fault Diagnosis via Interactive Multi-Task Learning. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 661–673. doi:10.1109/ASE63991.2025.00061

[17] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *ICWS*. 33–40. doi:10.1109/icws.2017.13

[18] Thong Hoang, Hoa Khanh Dam, Yasutaka Kamei, David Lo, and Naoyasu Ubayashi. 2019. DeepJIT: An end-to-end deep learning framework for just-in-time defect prediction. In *MSR*. 34–45. doi:10.1109/msr.2019.00016

[19] Thong Hoang, Hong Jin Kang, Julia Lawall, and David Lo. 2020. CC2Vec: Distributed representations of code changes. In *ICSE*. 518–529. doi:10.1145/3377811.3380361

[20] Jez Humble and David Farley. 2010. *Continuous delivery: reliable software releases through build, test, and deployment automation.* Addison-Wesley.

[21] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436. doi:10.48550/arXiv.1909.09436

[22] Yuze Jiang, Beijun Shen, and Xiaodong Gu. 2025. Just-in-time software defect prediction via bi-modal change representation learning. *Journal of Systems and Software* 219 (2025), 112253. doi:10.1016/j.jss.2024.112253
[23] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *ISSTA*. 437–440. doi:10.1145/2610384.2628055
[24] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2012), 757–773. doi:10.1109/tse.2012.70
[25] Rafael-Michael Karampatsis and Charles Sutton. 2020. How often do single-statement bugs occur? The ManySStuBs4J dataset. In *MSR*. 573–577. doi:10.1145/3379597.3387491
[26] Van-Hoang Le and Hongyu Zhang. 2022. Log-based anomaly detection with deep learning: How far are we?. In *ICSE*. 1356–1367. doi:10.1145/3510003.3510155
[27] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization. In *ISSTA*. 169–180. doi:10.1145/3293882.3330574
[28] Ziyang Liu, ChunHong Yuan, Hengjun Liu, Xiangyu Li, Shengrui Liu, Xiang Zhou, and Zhikui Tian. 2026. MSTDP: A multi-scale temporal deep learning framework for just-in-time software defect prediction with cross-attention fusion. *Journal of King Saud University – Computer and Information Sciences* 38, 2 (2026), 68. doi:10.1007/s44443-025-00401-y
[29] Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *ICLR*.
[30] Ben Maurer. 2015. Fail at Scale: Reliability in the Face of Rapid Change. *ACM Queue* 13, 8 (2015), 30–46. doi:10.1145/2838344.2839461
[31] Shane McIntosh and Yasutaka Kamei. 2018. Are fix-inducing changes a moving target? A longitudinal case study of Just-In-Time defect prediction. *IEEE Transactions on Software Engineering* 44, 5 (2018), 412–428. doi:10.1109/tse.2017.2693980
[32] Monique Louise Monteiro, George G Cabral, and Adriano L I Oliveira. 2025. CodeFlowLM: Incremental Just-In-Time Defect Prediction with Pretrained Language Models and Exploratory Insights into Defect Localization. arXiv:2512.00231. doi:10.48550/arXiv.2512.00231
[33] Edmilson Campos Neto, Daniel Alencar da Costa, and Uirá Kulesza. 2018. The impact of refactoring changes on the SZZ algorithm: An empirical study. In *SANER*. 380–390. doi:10.1109/SANER.2018.8330225
[34] Chao Ni, Wei Wang, Kaiwen Yang, Xin Xia, Kui Liu, and David Lo. 2022. The Best of Both Worlds: Integrating Semantic Features with Expert Features for Defect Prediction and Localization. In *ESEC/FSE*. 672–683. doi:10.1145/3540250.3549165
[35] Paolo Notaro, Jorge Cardoso, and Michael Gerndt. 2021. A survey of AIOps methods for failure management. *ACM Transactions on Intelligent Systems and Technology* 12, 6 (2021), 1–45. doi:10.1145/3483424
[36] OpenAI. 2024. GPT-4o System Card. OpenAI Technical Report.
[37] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. 2018. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3942–3951. doi:10.1609/aaai.v32i1.11671
[38] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. 2023. LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In *HPCC/DSS/SmartCity/DependSys*. 273–280. doi:10.1109/hpcc-dss-smartcity-dependsys60770.2023.00045
[39] Jiaxing Qi, Zhongzhi Luan, Shaohan Huang, Carol J Fung, Yuchen Wang, Aibin Wang, Hongyu Zhang, Hailong Yang, and Depei Qian. 2025. LogMoE: Lightweight Expert Mixture for Cross-System Log Anomaly Detection. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 330–341. doi:10.1109/ASE63991.2025.00035
[40] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *ICML*. 8748–8763.
[41] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2019. The impact of automated parameter optimization on defect prediction models. *IEEE Transactions on Software Engineering* 45, 7 (2019), 683–711. doi:10.1109/tse.2018.2794977
[42] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology* 28, 4 (2019), 1–29. doi:10.1145/3340544
[43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*. 5998–6008.
[44] Ke Wang and Zhendong Su. 2020. Blended, Precise Semantic Program Embeddings. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 121–134. doi:10.1145/3385412.3385999
[45] Xin Wang, Zhenhao Li, and Zishuo Ding. 2025. Defects4Log: Benchmarking LLMs for Logging Code Defect Detection and Reasoning. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1931–1942. doi:10.1109/ASE63991.2025.00161
[46] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. In *EMNLP*. 4542–4564. doi:10.18653/v1/2023.emnlp-main.68
[47] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie E Tan, Yitong Yieh, et al. 2020. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *ESEC/FSE*. 1556–1560. doi:10.1145/3368089.3417943
[48] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer. doi:10.1007/978-3-642-29044-2
[49] W Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A survey on software fault localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740. doi:10.1109/TSE.2016.2521368
[50] Qikai Wu, Xingqi Wang, Dan Wei, Bin Chen, and Qingguo Dang. 2025. Just-in-time software defect prediction method for non-stationary and imbalanced data streams. *Software Quality Journal* 33, 1 (2025). doi:10.1007/s11219-025-09711-w
[51] Pei Xiao, Chiming Duan, Minghua He, Tong Jia, Yifan Wu, Jing Xu, Gege Gao, Lingzhe Zhang, Weijie Hong, Ying Li, and Gang Huang. 2025. CoorLog: Efficient-Generalizable Log Anomaly Detection via Adaptive Coordinator in Software Evolution. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1119–1131. doi:10.1109/ASE63991.2025.00097
[52] Yifan Xiao, Shijie Li, and Yu Huang. 2026. CodeFault: Predicting Fault Risks from Code Changes via Multi-modal Learning — Replication Package. doi:10.5281/zenodo.21700762
[53] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *ESEC/FSE*. 807–817. doi:10.1145/3338906.3338931
[54] Xin Zhou, DongGyun Han, and David Lo. 2025. Bridging Expert Knowledge with Deep Learning Techniques for Just-In-Time Defect Prediction. *Empirical Software Engineering* 30, 37 (2025), 1–42. doi:10.1007/s10664-024-10591-0
[55] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *ISSRE*. 355–366. doi:10.1109/issre59848.2023.00071

Received 2026-03-26; accepted 2026-06-18